Conceitos Iniciais
Introdução ao Python

INE5402 A
INE5603 B
Prof. Luciana Rech

Algoritmo

- Algoritmo é a utilização de regras (conjunto de regras, ou conjunto de instruções) para definir ou executar uma tarefa específica ou para resolver um problema específico.

- Exemplo:
  - Receita culinária
  - Instruções de uso de um equipamento
  - Indicações de um instrutor de como estacionar um carro

Linguagens de Programação

- Uma **linguagem de programação** é um método padronizado para comunicar instruções para um computador. É um conjunto de regras sintáticas e semânticas usadas para definir um programa de computador.
- Permite que um programador especifique precisamente sobre quais dados um computador vai atuar, como estes dados serão armazenados ou transmitidos e quais ações devem ser tomadas sob várias circunstâncias.
- Linguagens de programação podem ser usadas para expressar algoritmos com precisão.

Linguagens de Programação

Compiladas: reúne o código fonte e o transforma, com um processo computacional chamado compilação, em um código de máquina. Este código de máquina é executado diretamente pelo seu Sistema Operacional.

Exemplos de linguagens compiladas: C, C++, Delphi, Pascal, Visual Basic

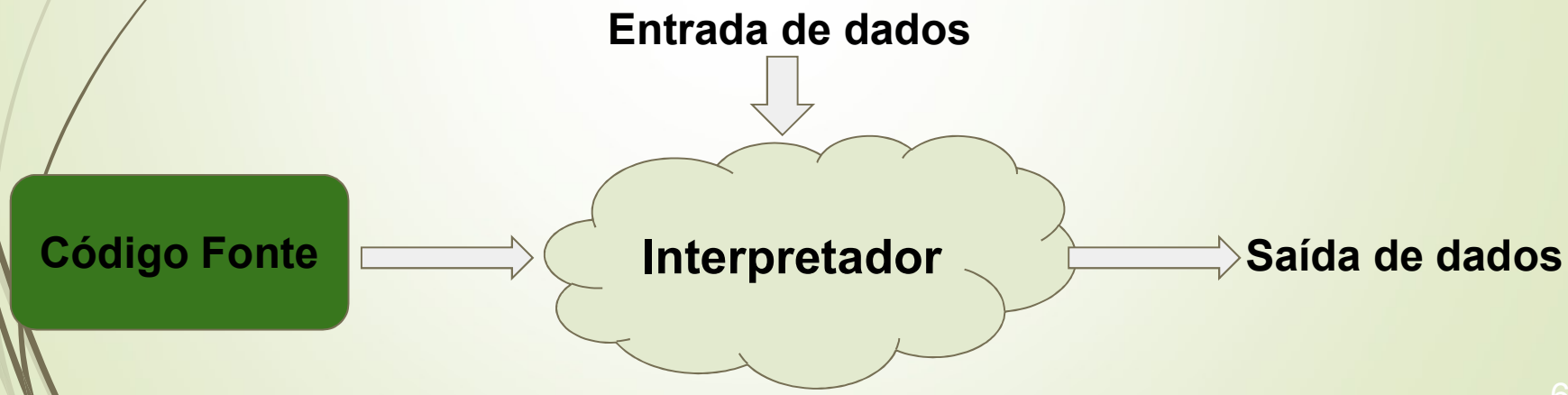



editor
PROGRAMA FONTE
texto
compilador
PROGRAMA OBJETO
linguagem de máquina
linkeditor
PROGRAMA EXECUTÁVEL
linguagem de máquina

Linguagens de Programação

Interpretada: o código fonte é executado diretamente por um programa de computador chamado interpretador e em seguida é executado pelo seu sistema operacional. As entradas de dados são feitas durante a execução do programa pelo interpretador.

Exemplos de linguagens interpretadas: Java, Ruby, C#, Javascript, PHP, Python, R,



Python

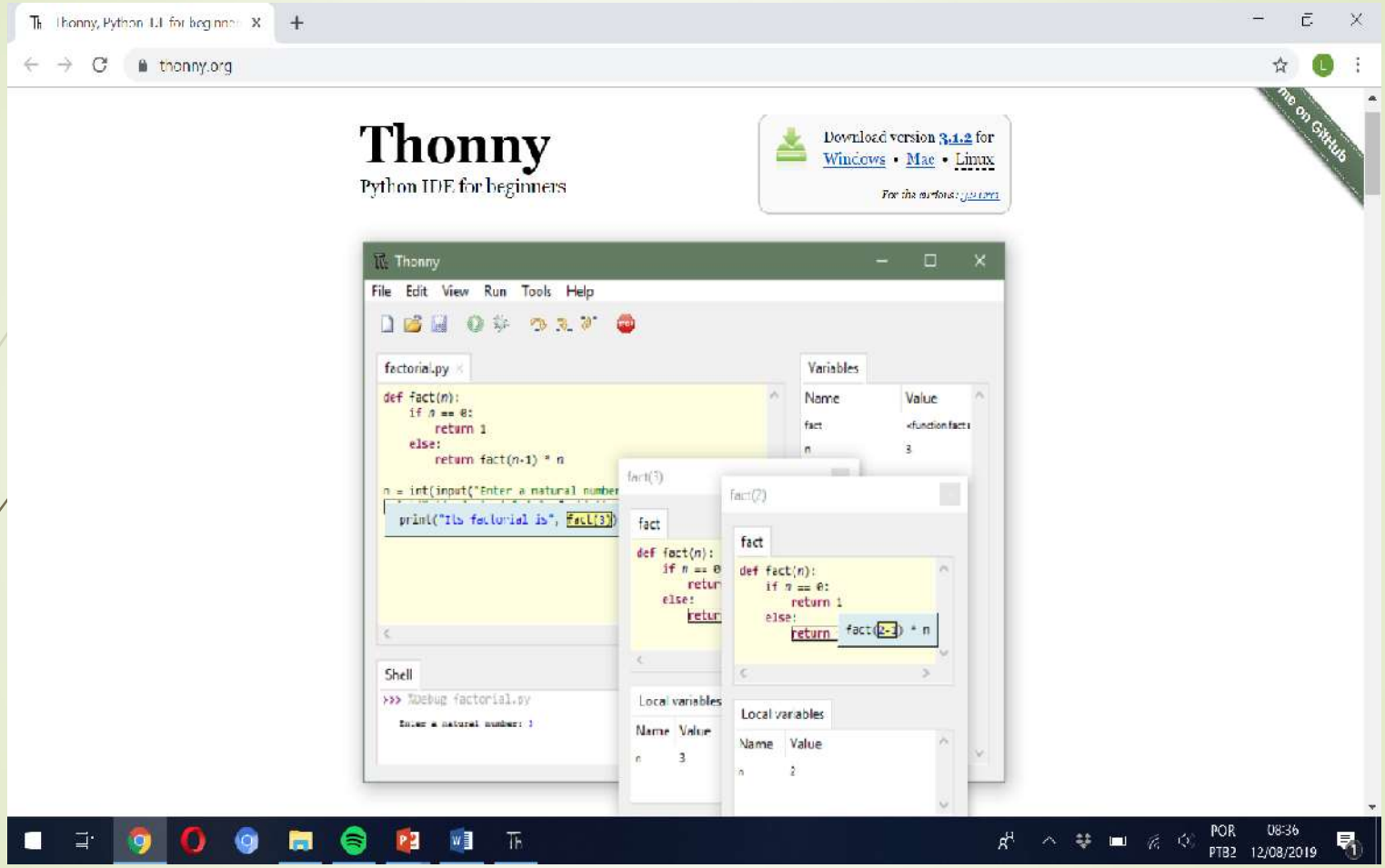
Instalar:

1) **Python**
www.python.org
- Versão 3.10.4
- IDLE (ambiente para rodar os arquivos .py sem precisar abrir o terminal)

2) **IDE**

- **Thonny** (IDE para iniciantes)
https://thonny.org/

- **Py Charm**
https://www.jetbrains.com/pycharm/


thonny.org
Thonny
Python IDE for beginners
Download version 3.1.2 for
Windows • Mac • Linux
Thonny
File Edit View Run Tools Help
factorial.py
Variables
Name Value
Shell
Local variables
Name Value
POR
PTB2
08:36
12/08/2019

Valores e Dados

Os valores (números e letras) são elementos fundamentais que um programa manipula.
Estes valores são normalmente transformados em dados de entrada e saída.

Por exemplo:

4

‘ Hello World! ’

Estes valores pertencem a um determinado tipo (ou classe) que comumente chamamos de Tipos de Dados (Data Types)
O valor 4 é um **inteiro** (int)
O valor “Hello World!” é um **texto** (String - Str)

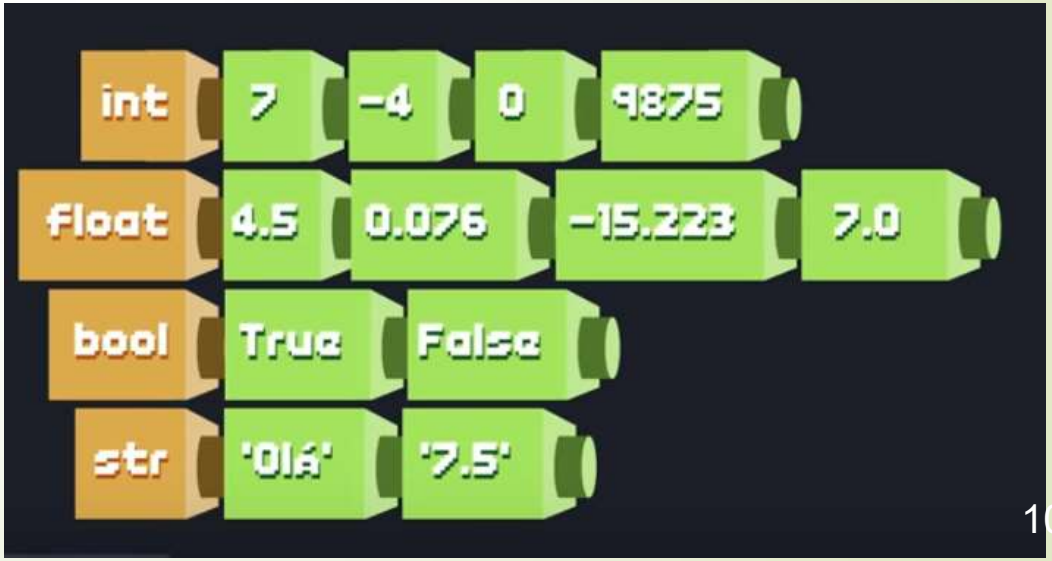
Tipo de Dados

INT (números inteiros)

FLOAT (números reais (com parte fracionária)

BOOL (True / False)

STR (caracter / string)



Variáveis

São elas que guardam os valores que os programas manipulam.

Por convenção, escrevemos as variáveis sempre com letras minúsculas.

As variáveis contém valores.

Para atribuir um valor a uma variável, usamos o símbolo **"="**

num = 13

text = "Hello World!"

Contantes

Identificadores que referenciam valores imutáveis.

Python, porém, não tem o conceito de constante como estrutura de linguagem.

Por convenção, usamos letra maiúscula para identificar que o valor daquela variável não deve ser alterado. Por exemplo: PI = 3,14159

Variáveis

Existem algumas regras básicas que regulam o batismo de variáveis em Python.

Estas regras são as seguintes:

- Nomes de variável só podem conter letras, dígitos e o caractere '_'
- Todo primeiro caractere deve ser sempre uma letra ou o caractere '_'
- Letras maiúsculas e minúsculas são consideradas caracteres diferentes, isto é, Python e C diferenciam a caixa das letras (maiúsculas e minúsculas)
- Palavras reservadas não podem ser usadas como nome de variáveis. Palavras reservadas são palavras usadas para indicar os comandos da linguagem, tipos de dados ou outras funções

É uma boa prática escolher nomes que indiquem o que ela representa

- exemplos: soma, mediaNotas, total, salario, nomeAluno, diaMes, ano, data_de_nascimento, telefone, cores...

Atribuição

Já sabemos atribuir valores a variáveis, mas é importante ressaltar que a variável que armazena o valor estará SEMPRE do lado esquerdo.

Portanto, a atribuição:

```
13 = n
```

lança um ERRO:

```
File "<interactive input>", line 1
SyntaxError: can't assign to literal
```

ERROS

São de três tipos e conhecê-los nos ajuda a compreender e resolver o problema/erro

1. Syntax: referem-se à estrutura do programa e as regras para escrevê-lo. Não permite a execução do seu código (exemplo, abrir e não fechar as aspas).
2. Runtime: o seu programa pode funcionar e o erro vai ocorrer durante a sua execução. Também chamado de exceção (exemplo, divisão por zero).
3. Semantic: não vai parar a execução do seu programa, mas o seu resultado será diferente do esperado. Muito difícil de identificar até você fazer testes conhecidos (erro de lógica matemática).

Comentários

São essenciais para os bons programadores.

O carácter usado para realizar comentários em Python é o "#"

Se o interpretador encontrar uma # em seu código, ele entenderá que é um comentário

```
# Este é apenas um comentário. Nenhuma tarefa é realizada pelo seu programa

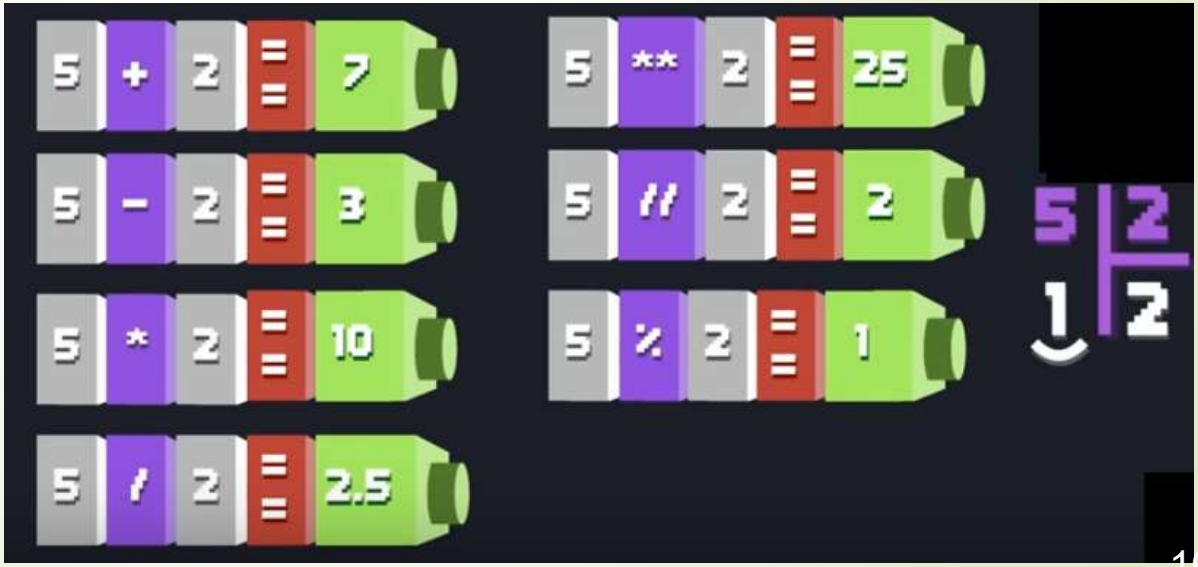
'''''
      .....

.......

'''''
```

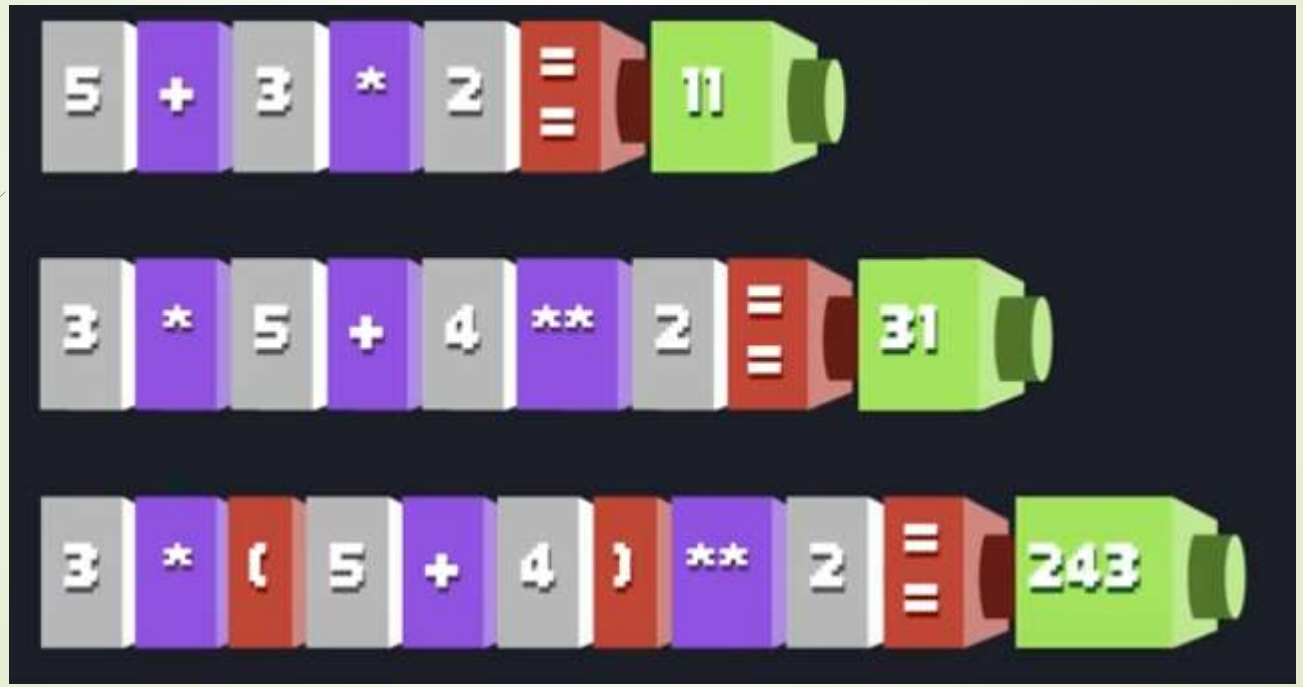
Operações Aritméticas

- SOMA (+)
- SUBTRAÇÃO (-)
- MULTIPLICAÇÃO (*)
- DIVISÃO (/)
- POTÊNCIA (**)
- DIVISÃO INTEIRA (//)
- RESTO DA DIVISÃO (%)



Ordem de Precedência entre Operações Aritméticas

1º)	**( )**	#parênteses
2º)	******	# potência
3º)	*** , / , // , %**	# multiplicação, divisão, divisão inteira e resto divisão
4º)	**+ , -**	# soma, subtração


5 + 3 * 2 == 11
3 * 5 + 4 ** 2 == 31
3 * (5 + 4) ** 2 == 243

Exemplos de uso de Operações Lógicas

```
# Exemplo de comparação de inteiros.
# Neste exemplo, 'a' é uma variável booleana,
# pois armazena o resultado de uma operação lógica.
a = 4 > 0 ❶
print(a)

# Strings são comparadas lexicograficamente, ou seja,
# em ordem alfabética, letra por letra.
b = "abacate" > "banana" ❷
print(b)

# Expressões lógicas podem envolver muitas variáveis.
# Elas podem também ser combinadas para produzir
# expressões mais complexas.
x = y = 7
print(x <= y and y <= x) ❸
```

Saída:

```
True
False
True
```

Leitura

Para lermos um valor digitado pelo usuário (teclado) e atribuí-lo a uma variável usamos a função input(“”):

n = input(“Digite um valor para ser armazenado na var n”)



O comando para escrever os dados de saída na tela é o print, mas para que o resultado seja o esperado devemos observar o tipo da variável que desejo imprimir.
Por exemplo:

```
n1 = input('digite um número:')
n2 = input('Digite outro número:')
s = n1+n2
print('Resultado da soma:',s)
```

Concatena
4 e 5

```
digite um número:4
Digite outro número:5
Resultado da soma: 45
```

correto:

```
1
2   n1 = int(input('digite um número:'))
3   n2 = int(input('Digite outro número:'))
4   s = n1+n2
5   print('Resultado da soma:',s)
6
```

```
digite um número:4
Digite outro número:5
Resultado da soma: 9
```

Conversões de tipos

```
# Converte inteiro para string
x = 10
s = str(x)
print(s)

# Converte float para inteiro
a = 2.5
b = int(a)
print(b)

# Converte inteiro para float
c = 2
d = float(c)
print(d)
```

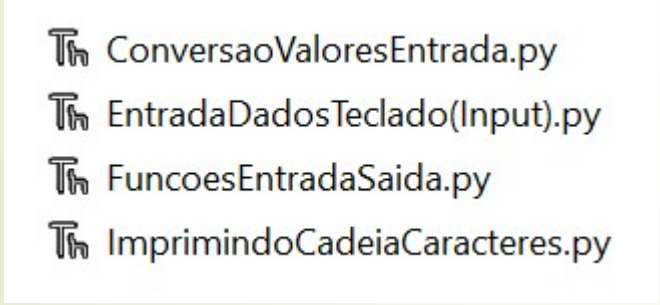
Saída:

```
10
2
2.0
```

Funções Básicas de Entrada e Saída

- *como mostrar (imprimir) dados*
- *como ler valores do teclado*
- *funções para conversão de valores*

Arquivos disponíveis no Moodle para executar no Thonny:



Exercícios:

1) Faça um algoritmo que calcule a área de um Triângulo (os valores da base e altura devem ser lidos do teclado)

2) Faça um algoritmo que leia o preço de um produto e mostre seu novo preço com 25% de desconto.

3) Faça um algoritmo que leia a idade de uma pessoa expressa em anos, meses e dias e escreva a idade dessa pessoa em dias. Considerar ano com 365 dias e mês com 30 dias.

4) Escreva um algoritmo para ler o número total de eleitores de um município, o número de votos brancos, nulos e válidos. Calcular e escrever o percentual que cada um representa em relação ao total de eleitores.

Seu programa deve verificar se a soma de brancos, nulos e válidos equivalem a 100% do número de eleitores que votaram.